

ESTRUCTURA Y DESARROLLO DE LA SOLUCIÓN

La solución para el sistema **n** se desarrolló utilizando una arquitectura robusta basada en el framework Spring Boot (Java), acoplada con un motor de plantillas dinámico (Thymeleaf) para el frontend y un sistema de seguridad avanzado (Spring Security).

A continuación, se detalla el flujo de desarrollo y la responsabilidad de cada componente dentro del proyecto:

1. Capa de Datos y Persistencia (Modelo y Repositorios)

- **Entidades de Dominio:** Se definieron las clases principales utilizando anotaciones de Jakarta/JPA (@Entity). Para este módulo, se modelaron las estructuras de Usuario y Rol, estableciendo una relación relacional (muchos usuarios poseen un único rol) para garantizar un **Acceso Seguro por Roles**.
- **Repositorios:** Se crearon interfaces que extienden de JpaRepository (como RolRepositorio y UsuarioRepositorio). Al utilizar Spring Data JPA, se automatizaron las operaciones CRUD de la base de datos MySQL sin necesidad de escribir código SQL manual, implementando consultas personalizadas derivadas como findByCorreo y findByNombre.

2. Capa de Seguridad y Configuración (SecurityConfig)

Para proteger el sistema y gestionar el flujo de autenticación, se implementó una clase de configuración global (@Configuration y @EnableWebSecurity):

- **Encriptación:** Se integró el componente BCryptPasswordEncoder para asegurar que ninguna contraseña se almacene o procese en texto plano, cumpliendo con los estándares de seguridad de la industria.
- **Filtros de Acceso (SecurityFilterChain):** Se definieron reglas explícitas para separar los recursos públicos de los privados. Las rutas críticas (como el dashboard) se bloquearon por completo, forzando a los usuarios no autenticados a redirigirse al login, mientras que los recursos estáticos (CSS, JS) y las pantallas de acceso (/login, /registro) se configuraron con acceso público (permitAll()).
- **Personalización del Formulario:** Se desvinculó el formulario automático de Spring para mapear y capturar los parámetros nativos de nuestro formulario HTML (correo y contraseña).

3. Capa de Control (Controladores)

El componente AuthController actúa como el intermediario entre la lógica del servidor y la interfaz gráfica del usuario:

- Mapea las peticiones HTTP para resolver y renderizar de forma ordenada las vistas estructuradas dentro del directorio de recursos (templates/auth/login, templates/auth/registro y dashboard).
- Controla el enrutamiento base, configurando la raíz del proyecto (/) para redireccionar de manera fluida los flujos de tráfico hacia la ventana de inicio de sesión.

4. Capa de Interfaz de Usuario (Frontend con Thymeleaf)

- La interfaz gráfica se diseñó con un enfoque limpio y profesional adaptado para micronegocios (PYMES), utilizando Bootstrap 5 para asegurar un comportamiento 100% responsivo en dispositivos móviles y ordenadores.
- Se utilizó Thymeleaf para interceptar el DOM del HTML y renderizar dinámicamente alertas del estado de seguridad del sistema (notificaciones de error por credenciales incorrectas o confirmaciones exitosas al cerrar sesión).
- Se inyectaron directivas de seguridad para la protección de datos, incluyendo campos ocultos automatizados para la validación de tokens, mitigando ataques de falsificación de peticiones en sitios cruzados.

Resumen del Flujo de Ejecución:

Cuando un usuario intenta acceder al sistema, Spring Security intercepta la petición. Si no ha iniciado sesión, lo redirige a la vista procesada por el AuthController. Al ingresar los datos en el formulario estructurado bajo Bootstrap, las credenciales viajan de forma segura hacia el contexto de seguridad, se comparan mediante algoritmos hash encriptados contra los registros existentes y, tras una validación exitosa, el sistema concede el acceso liberando de manera automática el entorno del Dashboard.